# Systèmes d'exploitation (Mons) – Exercices
# Chapitre 0.2 : Le Shell

Gauvain Devillez (gauvain.devillez@umons.ac.be)

Année académique 2026–2027 – Énoncés seuls

À chaque exercice est associé un nombre d'étoiles * qui correspond à son importance pour la matière. La signification est la suivante:

*** — exercice important qui devrait être fait avec soin.

** — exercice intéressant qui peut introduire de nouveaux concepts utiles ou approfondir la matière.

* — exercice moins central, à faire s'il reste du temps.

## 1 Introduction

L'*invite de commande* est une interface utilisateur textuelle. On retrouve des invites de commandes dans la plupart des systèmes d'exploitations ainsi que dans d'autres contextes comme l'IDLE de python ou encore certains jeux vidéo plus anciens.

Dans le contexte d'un système d'exploitation, elle permet d'interagir avec celui-ci. Si on sait s'en servir, l'invite de commande peut être un outil très puissant qui permet non seulement d'accéder facilement à tous les programmes disponibles, mais aussi de les combiner de manière complexe.

En pratique, l'invite de commande s'utilise au moyen d'un shell (ou interpréteur en ligne de commande) qui apporte ses fonctionnalités pour faciliter l'écriture de commandes. Il en existe beaucoup, mais le plus populaire est *Bash*. C'est celui que nous allons utiliser.

La première partie de ces exercices concernera l'utilisation de l'invite de commande. Souvent, ces shells supportent également un mode script, dans lequel ils exécutent un programme écrit dans un langage spécifique. Cela sera couvert dans la seconde partie.

## 2 Ligne de commande

Pour accéder à l'invite de commande, on doit passer soit par une *console* (virtuelle), soit par un *émulateur de terminal*. La console est une interface uniquement textuelle présente dans tous les systèmes Unix. L'émulateur de terminal est une application graphique qui se présente sous la forme d'une fenêtre dans laquelle on peut entrer du texte. Ce dernier étant plus pratique, nous allons nous en contenter.

Lorsque l'on ouvre un (émulateur de) terminal, on se retrouve face à une invite de commande. Il s'agit du shell. On peut alors entrer des commandes qui seront exécutées.

```
$ ls
file1
file2
$ pwd
/home/user
$
```

FIGURE 1 – Exemple d'utilisation du shell

Le shell s'exécute dans un répertoire (ou dossier) appelé *répertoire de travail* ou *répertoire courant* dont le nom et le chemin complet sont donnés par la commande `pwd`. La commande `ls`, va lister les fichiers dans le répertoire courant. On peut changer de répertoire avec la commande `cd`. On peut lui donner un chemin absolu (depuis la racine) ou relatif (depuis le répertoire courant). Dans les chemins relatifs, certains symboles ont un sens particulier :

`.` le répertoire courant,

`..` le répertoire parent (contenant le répertoire courant),

`~` le répertoire de l'utilisateur (répertoire home).

La plupart des commandes sont en réalité des exécutables (souvent programmés en C). Quand on exécute une commande, c'est le shell qui va chercher un programme de même nom et l'exécuter.

Pour avoir plus d'informations sur une commande, on peut utiliser la commande `man`. Celle-ci affiche la documentation de la commande passée en paramètre. On peut sortir de la documentation avec la touche `q` et effectuer une recherche avec la touche `/`. Il suffit alors d'entrer un texte à chercher suivi de la touche `enter`.

```
$ man man
MAN(1) Utilitaires de l'afficheur des pages de manuel MAN(1)

NOM
    man - Interface de consultation des manuels de référence du système

SYNOPSIS
    man [options de man] [[section] page ...] ...
    man -k [options d'apropos] expression_rationnelle ...
    man -K [options de man] [section] term ...
    man -f [options de whatis] page ...
    man -l [options de man] fichier ...
    man -w|-W [options de man] page ...

DESCRIPTION
    man est le programme de visualisation des pages de manuel. Chacun des
    arguments page, indiqué dans la ligne de commande de man, porte, en
    principe, le nom d'un programme, d'un utilitaire ou d'une fonction. La page
    de manuel correspondant à chaque argument est alors trouvée et affichée. Si
    une section est précisée alors man limite la recherche à cette section.
    Par défaut, il recherche dans toutes les sections disponibles en suivant
    un ordre prédéfini (voir DEFAULTS). Il n'affiche que la première page de
    manuel trouvée, même si d'autres pages de manuel existent dans d'autres
    sections.
```

FIGURE 2 – Exemple de `man`.

## 2.1 Les arguments

La plupart des commandes supportent différentes options. Il y a deux types d'options : les *options courtes* qui commencent par un seul tiret (e.g., `-l`) et les *options longues* qui commencent par deux tirets (e.g., `--all`). Certaines options peuvent également recevoir un argument (e.g., `--width=COLS`).

Comme montré dans la figure ci-dessous, il est possible de regrouper les options courtes avec un seul tiret. On peut également ignorer le symbole `=` pour les arguments des options.

```
$ ls -a -l
$ ls -al #identique a la commande precedente
$ ls --width=5
$ ls --width 5 #identique a la commande precedente
```

FIGURE 3 – Passage d'options aux commandes.

Certaines commandes prennent aussi des *arguments* qui ne sont pas des options. Par exemple `cat` permet d'afficher le contenu d'un ou plusieurs fichiers passés en argument.

Dans la plupart des shells, il existe un mécanisme permettant de facilement construire une liste d'arguments : la substitution. On distingue deux types différents qui sont l'expansion et le globbing.

L'*expansion* permet de générer une liste d'arguments, d'options ou même de texte sur base d'une spécification. Par exemple, la syntaxe suivante permet d'avoir toutes les paires de lettres : `{a..z}{a..z}`. On a deux expansions (`{a..z}`) et le shell va donc générer $26 \cdot 26 = 26^2$ éléments, un pour chaque valeur possible de chaque expansion. Les deux formats possibles sont visibles ci-dessous.

Le *globbing* est un concept proche des expressions régulières qui permet de choisir les fichiers à passer en paramètre via un format. On peut, par exemple, choisir uniquement les fichiers qui commencent par `log` via la syntaxe `log*`. Notez que l'interprétation est différente d'une expression régulière. Une liste de syntaxes est donnée dans la table 2 (page 3).

Table 1 – Expansions simples

| syntaxe | exemple |
|---|---|
| `{a,b,c}` → `a b c` | `img{a,b,c}.jpg` → `imga.jpg imgb.jpg imgc.jpg` |
| `{x..y}` → `x x+1 x+2 ... y` | `img{3..5}.jpg` → `img3.jpg img4.jpg img5.jpg` |

Table 2 – Globbing

| syntaxe | exemple | description |
|---|---|---|
| `?` | `log?.c` → `log1.c log2.c` | un seul caractère |
| `*` | `log*.c` → `log1.c logtest.c log.txt.c` | n'importe quelle séquence de caractères (même vide) |
| `[ab]` | `log[4ac].c` → `log4.c loga.c` | n'importe quel caractère entre les crochets |
| `[!ab]` | `log[!4ac].c` → `logb.c logd.c` | aucun des caractères entre les crochets |

### 2.1.1 Redirection

L'intérêt du shell est bien sûr de pouvoir combiner des commandes ensemble. Une première façon de faire est d'utiliser des fichiers temporaires. On peut rediriger la sortie des commandes (i.e., le texte affiché) vers un fichier et le texte d'un fichier vers une commande avec les opérateurs `<` et `>`. On appelle cela la *redirection*.

Il existe deux "fichiers" spéciaux [1] appelés la *sortie standard* (`stdout`) et la sortie d'*erreur standard* (`stderr`). Par défaut, le texte affiché par une commande est écrit dans `stdout` et les messages d'erreur dans `stderr`. Les deux sont affichés dans le terminal directement.

Ces deux sorties ont chacune un numéro qui peut être utilisé pour la redirection. La sortie standard a le numéro 1 et la sortie d'erreur standard a le numéro 2. Par défaut, les opérateurs `>` et `<` redirigent vers et depuis la sortie standard. Pour rediriger la sortie d'erreur standard, il suffit d'ajouter le numéro : `2>`. Pour rediriger vers une des deux sorties, il faudra ajouter le symbole `&` pour que le shell ne crée pas juste un nouveau fichier. Rediriger vers stderr se fait donc avec la syntaxe `>&2`. On peut rediriger `stderr` vers `stdout` (`2>&1`) et même avoir plusieurs redirections en même temps si elles redirigent des sorties différentes : `> fichier.txt 2>&1`. Attention, l'ordre est important.

```
$ pwd > monfichier.txt # ecrit le resultat dans monfichier.txt
$ sort < monfichier.txt # lit les donnees depuis le fichier au lieu de l'entree standard
$ cat f.txt 2>monfichier.txt # ecrit la sortie d'erreur (stderr) dans le fichier
$ cat f.txt 2>&1 # ecrit la sortie d'erreur (stderr) dans stdout (sortie standard)
$ cat f.txt >monfichier.txt 2>&1 # ecrit les deux sorties dans le fichier
$ cat f.txt 2>&1 >monfichier.txt # ecrit stdout dans le fichier et affiche stderr
$ cat f.txt >ok.txt 2>err.txt # ecrit les deux sorties dans des fichiers separes
```

Figure 4 – Redirection

Notez que, si le fichier existe déjà, l'opérateur `>` efface le contenu du fichier avant d'y écrire. Pour écrire à la fin du fichier sans écraser le contenu, on peut utiliser l'opérateur `>>` à la place.

Le fichier [2] `/dev/null` peut également être utilisé pour ignorer la sortie d'une commande. Tout texte envoyé vers `/dev/null` est simplement ignoré.

## 2.2 Gestion des jobs

Pour utiliser plusieurs commandes en même temps dans un même terminal, il est possible d'avoir des commandes en arrière-plan. C'est-à-dire, ne pas attendre que la commande se termine avant de lancer d'autres commandes. Pour le shell, chaque commande en train de s'exécuter est appelée un *job*. Lorsqu'on lance une commande, on peut ajouter `&` à la fin pour la lancer en arrière-plan. Le shell va alors immédiatement afficher l'invite de commande sans attendre la fin de ce job.

On peut voir la liste des commandes en arrière-plan via la commande `jobs`. Chaque job aura un identifiant numérique. Généralement, le premier job lancé qui n'est pas encore fini aura l'identifiant 1, le deuxième 2, etc. Il est possible de passer un job en arrière-plan en avant-plan (i.e., comme si on avait lancé la commande normalement), avec la commande `fg`. Cette commande prend un argument qui est l'identifiant du job précédé par `%`. Par défaut, si aucun argument n'est donné, c'est le job qui a le plus petit identifiant qui est choisi.

1. Ce ne sont pas vraiment des fichiers au sens classique du terme. On ne les trouvera pas sur le disque.
2. À nouveau, ce n'est pas un fichier au sens classique du terme.

Pour passer un job de l'avant-plan vers l'arrière-plan, il faut d'abord le suspendre (i.e., le mettre en pause) via le raccourci clavier `Ctrl+z`. Ensuite, on peut le relancer en avant-plan avec `fg` ou en arrière plan avec `bg`. Cette dernière s'utilise comme `fg`. Lorsqu'un job en arrière-plan se termine, l'information est affichée la prochaine fois que l'on lance une commande. Un exemple est visible dans la Figure 5, page 4.

```
$ ./slow &
[1] 11802
$ jobs
[1]+  En cours d'exécution    ./slow &
$ fg
./slow
<Ctrl+z>
[1]+  Stoppé                  ./slow
$ jobs
[1]+  Stoppé                  ./slow
$ bg
[1]+ ./slow &
$ pwd
/home/user
[1]+  Complété                ./slow
$ jobs
$
```

FIGURE 5 – Gestion des jobs

## 2.3 Combiner les commandes

Il est aussi possible de lancer plusieurs commandes l'une après l'autre et ainsi les combiner. Tout d'abord, on peut exécuter une commande uniquement si la précédente a réussi ou uniquement si elle a échoué.

La syntaxe `commande1 && commande2` ne va exécuter la deuxième commande que si la première a retourné 0. La syntaxe `commande1 || commande2` fait l'inverse (i.e., on exécute la deuxième commande uniquement si la première a retourné une valeur non-nulle).

En plus de cela, on peut *chaîner* des commandes. Le caractère `|`, placé entre deux commandes, va exécuter la première commande et envoyer la sortie de cette commande à l'entrée standard de la seconde. Par exemple, `ls | sort` va exécuter la commande `ls` et le texte qui aurait du être affiché sera envoyé à la commande `sort`.

On peut également utiliser une commande à la place d'un nom de fichier avec la syntaxe `<(commande)`. Cette syntaxe est utilisée comme argument à une autre commande. Voir Figure 6, page 5 pour des exemples.

```
$ rm fichier && echo "ok"
ok
$ rm fichier && echo "ok"
rm: impossible de supprimer 'fichier'
$ rm fichier2 || echo "erreur"
$ rm fichier2 || echo "erreur"
rm: impossible de supprimer 'fichier2'
erreur
$ grep "i" users.txt | sort
christian
christophe
hadrien
pierre
$ sort <(grep "i" users.txt) file.txt
bruno
christian
christophe
david
hadrien
pierre
```

FIGURE 6 – Exécution conditionnelle

# 3 Énoncés

Réalisez ces exercices dans un répertoire vide afin de limiter le risque de perdre vos fichiers en cas d'erreur. Vous pouvez créer un répertoire dans le répertoire `/tmp`, qui sera supprimé au redémarrage de l'ordinateur. Vous pouvez utiliser la liste de commandes fournie dans la Section 6.

* **Exercice 1** — A l'aide de la commande `cd`, se déplacer dans l'arborescence du système de gestion de fichiers. Utiliser des chemins absolus, relatifs ainsi que `.` et `..` . Vérifier le positionnement dans l'arborescence grâce à `pwd`.

*** **Exercice 2** — A l'aide d'un éditeur de texte, créer deux ou trois fichiers texte contenant quelques lignes.

* **Exercice 3** — Visualiser les contenus des fichiers créés à l'aide des commandes `cat` et `less`.

*** **Exercice 4** — À l'aide de `mkdir`, `cd`, `cp`... créer une petite arborescence dans le répertoire privé, contenant des copies des fichiers créés.

*** **Exercice 5** — Utiliser `mv` pour changer les noms de fichiers et de répertoires.

*** **Exercice 6** — Utiliser `cp` "récursif" pour réaliser une nouvelle copie de l'arborescence créée.

*** **Exercice 7** — Tentez d'utiliser la communication par tube (`|`) avec les commandes `head` et `tail` pour afficher les lignes de 4 à 7 de `/etc/passwd`.

*** **Exercice 8** — Placez-vous dans un répertoire vide. Créez, à l'aide de la commande `touch`, 100 fichiers dont le nom est de la forme "fichier`xy`.txt" où `x` et `y` sont des chiffres de 0 à 9. Utilisez les expansions.

** **Exercice 9** — Créez des fichiers dont le nom commence par "result" suivi d'une des lettres `a`, `b`, `c`, `f`, `g`, `h`. Créez un fichier par lettre.

*** **Exercice 10** — Créez un nouveau répertoire vide et déplacez, avec `mv`, les fichiers "fichier`xy`.txt" ne contenant pas le chiffre 3. Utilisez le globbing.

* **Exercice 11** — En une seule commande, affichez les lignes de `/etc/passwd` qui concernent des utilisateurs dont le shell n'est pas Bash.

** **Exercice 12** — En une seule commande, affichez la liste des shells des utilisateurs de `/etc/passwd`. Chaque utilisateur ne doit être affiché qu'une seule fois. Utilisez `cut`, `uniq` et `sort`.

*** **Exercice 13** — Comptez le nombre de lignes de `/etc/passwd` contenant le mot "`bash`". En plus des commandes ci-dessus, utilisez `wc`.

** **Exercice 14** — La commande `find / -name "bash*"` permet de rechercher, dans toute l'arborescence du système de gestion de fichier, tous les fichiers dont le nom débute par `bash`. Cette commande affichera des résultats et des messages d'erreurs. Il vous est demandé, à l'aide de redirections et/ou de communications par tube, de lancer la commande de telle sorte que les résultats disparaissent, par contre les messages d'erreurs doivent être écrits dans un fichier `erreurs.txt`.

** **Exercice 15** — Modifiez la commande `find` précédente pour trouver tous les fichiers dont le nom commence par "test". Utilisez la redirection pour cacher la sortie et les messages d'erreur de la commande. Mettez cette commande en arrière-plan et vérifiez son statut. Ensuite, repassez-la en avant-plan et arrêtez-la (Ctrl+c).

* **Exercice 16** — Lancez la même commande que l'exercice précédent, mais directement en arrière-plan. Stoppez-la après quelques minutes.

# 4 Scripts

Une commande peut devenir assez longue lorsqu'elle combine beaucoup d'instructions. Il est donc pratique de pouvoir écrire ces commandes dans un script afin de pouvoir les réutiliser. Les scripts permettent également d'utiliser des structures de contrôle comme les boucles ou les conditions.

## 4.1 Écrire et exécuter un script

Pour créer un script, il suffit d'écrire des commandes dans un fichier. Cependant, il est fortement recommandé de commencer ce fichier par la ligne `#!/bin/bash`. Cette ligne spéciale, appelée *shebang*, indique au système d'exploitation quel interpréteur utiliser pour exécuter le script. Sans elle, le système pourrait utiliser un shell par défaut (comme `sh`) qui ne supporte pas toutes les fonctionnalités de bash utilisées dans ce cours.

En général, ce fichier aura l'extension `.sh`. On peut alors l'exécuter comme on le fait pour un exécutable produit par le compilateur C.

Cependant, il est fort probable que le shell produise une erreur disant que le script n'est pas exécutable ou qu'une permission n'a pas été accordée. Sous GNU/Linux, les *permissions* sont un mécanisme de sécurité.

Il y a trois grandes permissions : la lecture, l'écriture et l'exécution. Ces trois permissions peuvent être données à trois niveaux : le propriétaire, le groupe et "les autres".

Le *propriétaire* est le propriétaire du fichier. Le *groupe* est le groupe du fichier. Les groupes sont simplement des groupes d'utilisateurs. Tous les utilisateurs peuvent appartenir à un ou plusieurs groupes. Si un utilisateur appartient au groupe associé à un fichier, il aura les permissions spécifiées pour ce groupe. Notez que rien n'oblige le propriétaire à appartenir au groupe du fichier. Enfin, "les autres" représente simplement les utilisateurs qui ne sont ni le propriétaire du fichier, ni dans son groupe. La commande `ls -l` permet de voir le propriétaire et le groupe d'un fichier ainsi que les permissions.

La commande `chmod` permet de changer les permissions d'un fichier. Elle reçoit en argument un ou plusieurs changements à effectuer et une liste de fichiers. Ces changements sont de la forme `niveau[+-]permission`. Pour un niveau (e.g., propriétaire), on peut ajouter (`+`) ou retirer `-` une permission (e.g., lecture). Les niveaux et les permissions sont représentés par des lettres. Les niveaux sont :

**u :** le propriétaire,

**g :** le groupe du fichier,

**o :** les "autres",

**a :** les trois niveaux en même temps.

Et les permissions sont :

**r :** lecture,

**w :** écriture,

**x :** exécution.

Pour rendre un script exécutable par tout le monde, la commande serait donc : `chmod a+x script.sh`.

## 4.2 Variables et expressions

Comme beaucoup de langages de programmation, le shell propose des variables. Celles-ci sont déclarées par une simple égalité et on y accède en utilisant l'opérateur `$`. Lorsque l'on utilise cet opérateur, on peut également ajouter des accolades `${var}`). On peut effacer une variable (i.e., vider son contenu) avec la commande `unset`. Il est aussi possible de stocker le résultat d'une commande avec la syntaxe `var=$(commande)`.

```
a=5 # Notez l'absence d'espaces!
echo $a
b=$(ls) # b contiendra le résultat de ls
echo $ab # variable ab non définie, rien n'est affiché
echo ${a}b # affiche 5b (a=5)
unset a # La variable n'est plus définie
```

FIGURE 7 – Utilisation de variables

Certaines variables sont appelées des *variables d'environnement*. Il s'agit de variables qui seront disponibles pour tous les sous-processus. Par exemple, une variable d'environnement définie dans le shell sera disponible pour toutes les commandes. Une variable d'environnement très utilisée est `PATH` qui contient une liste de répertoires dans lesquels le shell ira chercher les exécutables des commandes. Dans Bash, celle-ci et d'autres sont déclarées

dans un fichier `.bashrc` qui se trouve dans le répertoire de l'utilisateur. Ce fichier est exécuté à chaque lancement de Bash. Pour transformer une variable en variable d'environnement, on utilise la commande `export`.

En Bash, tout texte est considéré comme une chaîne de caractères. Cependant, certaines parties du texte peuvent être interprétées par le shell et donner un résultat différent. Par exemple, la présence d'un espace dans un texte peut inciter Bash à considérer qu'il s'agit de plusieurs arguments ou mots distincts.

Si on utilise les double-guillemets (`"`), le texte est considéré comme un seul élément et pas comme un tableau. Par contre, les variables (avec le caractère `$`)) seront remplacées par leur valeur. Les simple-guillemets (`'`) empêchent le remplacement de variables.

Comme mentionné plus haut, on peut définir des tableaux. Un tableau est un ensemble d'éléments séparés par un des caractères de la variable d'environnement `IFS`. Par défaut, celle-ci contient l'espace, le retour à la ligne et la tabulation. On peut également utiliser le globbing pour construire un tableau contenant les noms de fichiers choisis. Une fois le tableau défini, plusieurs opérateurs permettent d'y accéder :

**`a=(c d)`** création d'un tableau à deux éléments

**`$a[0]`** accès au premier élément

**`$a[@]`** toutes les valeurs (utile pour une boucle)

**`$a[*]`** toutes les valeurs en une seule entité

**`$#a[@]`** nombre d'éléments

**`$!a[@]`** liste des indices

**`a=( dossier/*.txt )`** création d'un tableau de noms de fichiers

Par défaut, le contenu de toute variable est considéré comme du texte. On peut effectuer des calculs avec la syntaxe `$((calculs))`. Dans cette syntaxe, les opérateurs classiques existent (addition, soustraction...). On peut utiliser des variables existantes mais sans le symbole `$`.

```
$ a=world
$ echo $a
world
$ echo "hello $a"
hello world
$ echo 'hello $a'
hello $a
$ b=(hello world)
$ echo $b
hello
$ echo ${b[1]}
world
$ echo ${b[@]}
hello world
$ a=5
$ echo $a+1
5+1
$ echo $((a+1))
6
```

FIGURE 8 – Variables, tableaux et arithmétique

#### L'importance des guillemets

Une erreur très courante en Bash consiste à oublier les guillemets autour des variables. Par défaut, le shell effectue ce qu'on appelle le *word splitting* : il découpe le contenu d'une variable en plusieurs arguments s'il y trouve des espaces. Cela peut être dangereux, notamment lors de la manipulation de fichiers (voir Figure 9).

Il est donc fortement recommandé de **toujours** entourer vos variables de doubles guillemets (i.e., `"$variable"`). Cela permet aussi d'éviter des erreurs de syntaxe si la variable est vide ou si elle contient des caractères spéciaux (comme `*`).

### 4.3 Fonctions

La définition de fonctions est un peu différente en Bash par rapport à d'autres langages. On ne liste pas les arguments de la fonction. À la place, on accède aux arguments via les variables `$1`, `$2`... La syntaxe utilisée pour

```
fichier="mon cv.txt"

# ERREUR : Bash voit deux arguments : "mon" et "cv.txt"
rm $fichier
# Résultat : "rm: impossible de supprimer 'mon': Aucun fichier..."

# CORRECT : Bash voit un seul argument : "mon cv.txt"
rm "$fichier"
```

FIGURE 9 – L'impact des guillemets sur les espaces.

les tableaux peut aussi être utilisée sans donner de nom de variable afin de récupérer le nombre d'arguments : `$#`, `$@`...

Pour appeler une fonction, on n'utilise pas de parenthèses, mais liste les arguments séparés par des espaces, comme pour les commandes. On peut voir les fonctions comme des commandes spéciales. Un script est également une commande spéciale et utilise la même syntaxe que les fonctions pour ses arguments.

Par défaut, les variables sont globales. Pour définir une variable locale, il faut ajouter le mot-clé `local`. Notez que c'est le cas pour toutes les variables, incluant les variables déclarées dans les boucles !

Pour renvoyer une valeur, on va généralement utiliser la commande `echo`. Comme pour les commandes, le résultat d'une fonction est ce qu'elle affiche (ou devrait afficher) dans le terminal. Un exemple de script utilisant une fonction est donné dans la Figure 10, page 9.

```
function mafonction {
    echo "premier argument: $1"
    echo "nombre d'arguments: $#"
    local num=$2
    echo $((num+1))
}

mafonction hello 4
echo $num # n'affiche rien
echo $1 # Premier argument du script
```

FIGURE 10 – Les fonctions

## 4.4 Structures de contrôle

Les expressions conditionnelles utilisent une syntaxe particulière. Dans le cas de comparaisons numériques, on utilise la syntaxe pour les expressions arithmétiques, sans le `$` (e.g., "`((1<2))`"). Pour les autres conditions, on utilise des double crochets (e.g., "`[[ condition ]]`"). On peut alors comparer des chaînes de caractères en utilisant les opérations classiques (`<`, `>`, `==`...). Notez qu'il est important d'ajouter des espaces autour de la condition. L'instruction `[[condition]]` ne fonctionnera pas et devrait être `[[ condition ]]`.

Les conditions peuvent porter sur des chaînes de caractères (e.g., égalité, ordre alphabétique...) ou encore des propriétés de fichiers via des opérateurs spéciaux.

```
# Attention: espaces autour de l'expression!
[[ "allo" < "biking" ]] # vrai
[[ "$1" == "quit" ]]
[[ 21 < 3 ]] # vrai, ordre alphabétique
(( 21 < 3 )) # faux
[[ "$1" == "quit" ]] || (( n > 10 ))
[[ -e "fichier.text" ]] # Est-ce que le fichier existe
[[ -d "fichier.text" ]] # Est-ce que c'est un dossier
[[ -f "fichier.text" ]] # Est-ce que c'est un fichier régulier
```

FIGURE 11 – Expressions conditionnelles

Avec ces conditions, on peut maintenant utiliser des instructions conditionnelles (`if`). Celles-ci, tout comme le `while` et le `for`, fonctionnent de la même façon que dans les autres langages. La syntaxe, cependant, impose un mot-clé pour signaler la fin de l'instruction et un mot-clé pour séparer la condition du corps de l'instruction.

```
if condition; then # Ne pas oublier le then!
    code
else if condition
then # Le ; peut être remplacé par un retour à la ligne
    code
fi # Signale la fin du si

for var in ${liste[@]}; do
    code
done

for (( i=0; i<n; i++ )); do
    code
done

while condition; do
    code
done
```

FIGURE 12 – Structures de contrôle

# 5 Énoncés (scripts)

### Exercice 17

Créer un script `affiche` qui prend des noms de fichiers en paramètres. Pour chaque paramètre, trois cas sont possibles :

— Le fichier n'existe pas. Afficher le nom du fichier suivi de "non existant".
— C'est un fichier ordinaire. Afficher le nom du fichier ainsi que tout son contenu.
— C'est un répertoire. Afficher le nom du répertoire et les noms des fichiers qu'il contient.

### Exercice 18

Étant donné un ensemble de fichiers texte contenant chacun une liste de nombres entiers (un nombre par ligne), écrire le programme `max` qui affiche le plus grand de ces nombres et affiche le nom du fichier qui contient ce nombre (si plusieurs fichiers contiennent ce nombre, afficher uniquement le nom d'un de ces fichiers). Les noms des fichiers sont passés en paramètres au programme `max`.

### Exercice 19

Créer le script Bash `renomme` qui, appelé sous la forme `renomme pref fichier file fich2`, renomme les fichiers `fichier`, `file` et `fich2` en leur donnant respectivement les noms `pref1`, `pref2` et `pref3`.

Le premier paramètre constitue donc un préfixe des nouveaux noms, les autres paramètres sont les références des fichiers à renommer. Ils sont renommés en leur donnant, comme suffixe, 1, 2, 3... Seuls les fichiers ordinaires existants seront renommés de cette façon. Si un nom de fichier passé en paramètre n'existe pas ou ne correspond pas à un fichier ordinaire, il ne doit pas y avoir de rupture dans la séquence des numéros.

Si un nouveau nom de fichier existe déjà, alors le numéro d'ordre est automatiquement incrémenté jusqu'à ce que le nouveau nom n'existe pas (par exemple, si `pref3` existe déjà, on l'appellera `pref4` si cela est possible, ou `pref5` si cela est impossible, etc).

### Exercice 20

Concevoir le script `StructArbo` qui effectue une copie récursive vide d'une arborescence. Le premier paramètre passé au script est le répertoire racine de l'arborescence à copier, le second paramètre est le répertoire de destination.

L'arborescence doit être complètement recréée : tous les répertoires, sous-répertoires... doivent se trouver dans la copie, mais les autres fichiers (fichiers réguliers) ne doivent pas s'y trouver. Seule doit être donc conservée la structure arborescente.

Cependant, dans la copie, chaque répertoire doit contenir le fichier `index.txt` qui liste les fichiers présents dans le répertoire source correspondant à l'exception de ses sous répertoires. Elle doit contenir, pour chaque fichier :

— le nom du fichier,
— la taille du fichier (commande `du`),
— le type `mime` du fichier (commande `file`).

Lorsque le répertoire source ne contenait aucun lien excepté ses sous-répertoires, le fichier `index.txt` correspondant contiendra un message qui le signale. Si le répertoire source n'est pas accessible, le fichier `index.txt` correspondant le signalera également.

### Exercice 21

Développer le script Bash `filespy` qui analyse récursivement, toutes les 3 secondes et indéfiniment, une arborescence à la recherche de fichiers dont les noms ont des extensions déterminées. Pour chaque fichier détecté (c'est-à-dire dont le nom possède une des extensions passées en paramètres), sa localisation sera affichée sur la console. Ensuite, le fichier sera copié vers un répertoire archive, tout en conservant l'organisation récursive des fichiers.

Par exemple, l'appel :

```
$ filespy /tmp/depot /tmp/archive .mp3 .crack .pwd
```

doit analyser, toutes les 3 secondes, l'arborescence enracinée en `/tmp/depot` à la recherche des noms de fichiers se terminant par `.mp3` ou `.crack` ou encore `.pwd`. Le nom de chaque fichier trouvé sera affiché. Le fichier sera alors copié dans l'arborescence enracinée en `/tmp/archive`.

Remarques :

— Le script ne peut créer aucun répertoire inutile.
— La commande Bash : `sleep n`, permet de suspendre l'exécution durant exactement `n` secondes.
— Bonus : ne copiez un fichier que s'il a été modifié depuis la dernière copie.

# 6 Commandes utiles

| commande | description | options utiles |
|---|---|---|
| `man` | Affiche la documentation d'une commande. | |
| `pwd` | Affiche le chemin du répertoire courant. | |
| `cd` | Change le répertoire courant. | |
| `ls` | Liste les fichiers dans le répertoire courant. | **`-a`:** liste également les fichiers commençant par un point.<br>**`-l`:** affiche plus d'information comme le nom du propriétaire |
| `echo` | Affiche un texte dans le terminal. | |
| `cat` | Affiche le contenu de fichiers. | |
| `head` | Affiche les 10 premières lignes d'un texte. | **`-n NOMBRE`:** Affiche les NOMBRE premières lignes. Si NOMBRE commence par un `-`, affiche toutes les lignes sauf les NOMBRE dernières. |
| `tail` | Affiche les 10 dernières lignes d'un texte. | **`-n NOMBRE`:** Affiche les NOMBRE dernières lignes. Si NOMBRE commence par un `+`, affiche à partir de la ligne NOMBRE (incluse). |
| `grep` | Affiche uniquement les lignes contenant un texte. | **`-v`:** Sélectionne les lignes ne contenant pas le texte.<br>**`-i`:** Ignore la casse (i.e., majuscule ou minuscule).<br>**`-r`:** Cherche aussi dans les sous-répertoires. |
| `sort` | Trie les lignes dans l'ordre alphabétique. Peut gérer des fichiers très larges. | **`-n`:** Utilise l'ordre numérique. |
| `uniq` | Retire les lignes identiques consécutives. Pour enlever tous les doublons, il faut d'abord trier. | **`-c`:** Ajoute le nombre de répétitions consécutives de chaque ligne. |
| `cut` | Découpe chaque ligne en colonnes numérotées à partir de 1 et ne garde que celles indiquées par l'option `-f`. | **`-f=LISTE`:** Indices des colonnes à garder. Les colonnes doivent être séparées par des virgules.<br>**`-d=DELIM`:** Le caractère à utiliser pour séparer les colonnes. Par défaut, le caractère de tabulation est utilisé. |
| `paste` | Concatène les lignes de plusieurs fichiers dans l'ordre. | **`-d=DELIM`:** Caractère à ajouter entre les lignes des différents fichiers. |
| `file` | Affiche le type de fichier, indépendamment de son extension. | |
| `touch` | Crée un fichier s'il n'existe pas encore. | |
| `mv` | Déplace (copie puis supprime) un fichier ou un répertoire. Peut être utilisé pour renommer. | |
| `mkdir` | Crée un répertoire s'il n'existe pas encore. | **`-p`:** Crée aussi les parents du répertoire s'ils n'existent pas. |
| `rmdir` | Supprime un répertoire vide. | **`-p`:** Supprime aussi les répertoires parents vides. |

| commande | description | options utiles |
|---|---|---|
| `rm` | Supprime un fichier. Attention, il n'est pas possible de récupérer le fichier ensuite. | `-r`: Supprime un répertoire et tout ce qu'il contient. |
| `sed` | Applique des expressions régulières pour chercher et, optionnellement, remplacer dans un texte. | |
| `find` | Cherche des fichiers dans une arborescence. Beaucoup d'options. | |
| `less` | Facilite la visualisation d'un texte via la pagination. | |
| `shift` | Décale les arguments (`$2` devient `$1` et ainsi de suite.) | |
| `basename` | Retourne le nom du fichier sans le chemin (e.g., `/home/user/fichier` → `fichier`). | |
| `wc` | Compte le nombre de lignes, de mots, de caractères et/ou d'octets (*bytes*) de l'input (`wc -l` pour compter le nombre de lignes). | |

# 7 Exemples de scripts

```
function fact {
    local n=$1
    if ((n==0)); then
        echo 1
    else
        local f=$(fact $((n-1)))
        echo $((n*f))
    fi
}

n=$1
fact $n
```

FIGURE 13 – Calcul de la factorielle

```
function list {
    local name="$1"
    local file # Important!
    if [[ -d "$name" ]]; then
        for file in "$name"/*; do
            list "$file"
        done
    elif [[ -e "$name" ]]; then
        echo "$name"
    fi
}

for file in $*; do
    list "$file"
done
```

FIGURE 14 – Affiche les fichiers réguliers récursivement